

# 读得懂，再记得住

使用说明与六组学习内容对应关系

**本版改了什么** 由14页短提纲扩展为40道逐题讲解。每题提供独立页码、概念拆解、原因或流程、例子、误区和带简答的自测；不要求逐字背诵。

**页码怎样用** “先读”是最直接的原书位置，“再查”是补背景或深入的位置。均为你提供提供的322页《Cortex-M3权威指南》的PDF物理页，不是本修订版页码。每道题内的小检查题沿用本题参考页。

**没有对应章节怎么办** FreeRTOS API、M0+、M4/M7和具体STM32电源实现并非旧指南完整覆盖的内容。相关题明确区分“指南背景页”和“官方直接入口”，不制造虚假页码。0编号的可点击来源见最后一页。

**按你原来的六组学** ①基础、寄存器、双栈、启动：Q01~Q11；②存储与访问：Q12~Q16、Q29~Q31；③异常/NVIC/现场：Q17~Q25、Q34；④RTOS衔接：Q06~Q07、Q26~Q28、Q33；⑤低功耗/调试：Q24~Q25、Q32~Q35；⑥汇编/链接查用：Q10~Q11、Q36~Q38。Q39~Q40只补跨核差异。

**如何判断学会** 先读正文，用自己的话解释机制，再遮住最后一段回答自测。答不出就按“先读”翻书；仍不清楚再看“再查”。A要求会解释和应用，B理解用途与限制，C仅按岗位深挖。

**重要边界** 本文是原创讲解与纠错，不是原书全文替代，也不是官方认证教材。示例用于理解，不代表已在你的板上实测；不要求学习Linux。原始华清PDF与权威指南均保留。

# 题目导航 1/2

点击题目可跳转；右侧是本修订版页码，不是指南页码

|                                   |    |
|-----------------------------------|----|
| Q01 内核、架构和MCU，究竟分别是什么？            | 4  |
| Q02 M3为什么是32位，却能执行16位指令？          | 5  |
| Q03 流水线、哈佛结构和总线有什么联系？             | 6  |
| Q04 R0~R15、xPSR和函数调用怎样配合？         | 7  |
| Q05 Thread、Handler与特权级是怎样切换的？     | 8  |
| Q06 MSP和PSP怎样工作？异常时现场存哪？          | 9  |
| Q07 满递减栈、32字节现场和8字节对齐怎么区分？        | 10 |
| Q08 复位后CPU如何找到第一条指令？              | 11 |
| Q09 向量表、异常号、IRQ号与VTOR怎样对应？        | 12 |
| Q10 Reset_Handler到main，中间做了什么？    | 13 |
| Q11 如何用链接脚本与Map判断Flash/RAM占用？     | 14 |
| Q12 存储映射与Normal/Device属性为什么重要？    | 15 |
| Q13 大小端与非对齐访问，分别影响什么？             | 16 |
| Q14 位带为什么能改单个位？能替代互斥锁吗？           | 17 |
| Q15 LDREX/STREX如何避免读改写丢失？         | 18 |
| Q16 MPU与MMU有什么区别？MPU能防止什么？        | 19 |
| Q17 异常、中断、Fault分别是什么关系？           | 20 |
| Q18 NVIC优先级数值越小，为什么越紧急？           | 21 |
| Q19 抢占优先级与子优先级怎样决定执行顺序？           | 22 |
| Q20 Enable、Pending、Active为何必须分开看？ | 23 |

## 题目导航 2/2

点击题目可跳转；右侧是本修订版页码，不是指南页码

|                                            |    |
|--------------------------------------------|----|
| Q21 PRIMASK、BASEPRI、FAULTMASK怎样屏蔽？         | 24 |
| Q22 异常进入、自动入栈与EXC_RETURN如何连起来？             | 25 |
| Q23 尾链、晚到与普通中断嵌套有何不同？                      | 26 |
| Q24 中断延迟为什么不能只算12个周期？                      | 27 |
| Q25 SysTick、主频与RTOS Tick如何计算和区别？           | 28 |
| Q26 SysTick、SVC、PendSV怎样完成任务切换？            | 29 |
| Q27 FromISR与xHigherPriorityTaskWoken到底管什么？ | 30 |
| Q28 临界区为什么要恢复旧状态，而不是直接开中断？                 | 31 |
| Q29 volatile、原子性与数据一致性为什么不是一回事？            | 32 |
| Q30 DMB、DSB、ISB分别解决什么问题？                   | 33 |
| Q31 DMA为什么会共享缓冲区出问题？                       | 34 |
| Q32 WFI/WFE与Sleep、Stop、Standby怎样联系？        | 35 |
| Q33 RTOS Tickless与裸机RTC定时唤醒一样吗？            | 36 |
| Q34 HardFault发生后，如何拿到可信现场？                 | 37 |
| Q35 断点、观察点、寄存器和仪器分别怎样用？                    | 38 |
| Q36 LDR、STR和LDR =地址，怎样读懂？                  | 39 |
| Q37 PUSH/POP、BL/BX怎样支撑函数调用？                | 40 |
| Q38 MRS/MSR与MOV、LDR/STR有什么区别？              | 41 |
| Q39 从M3换到MSPM0，必须补哪些区别？                    | 42 |
| Q40 其他ARM学到哪？M4/M7与旧ARM怎样取舍？               | 43 |

## Q01 内核、架构和MCU，究竟分别是什么？

A 必学 | 一题一页，先理解再复述

**先读指南** 第2章p21~28：先看处理器、寄存器、NVIC与存储系统的分工。

**没懂再查** 第1章p13~19；第6章p99~104：补架构背景和内核接口。

**先把对象分开** 架构规定软件能看到的规则，例如有哪些指令、寄存器和异常行为；内核是实现这些规则的处理器设计；MCU则把内核、Flash、SRAM、时钟、电源控制和外设整合成可用芯片。三者不是同一层的名字。

**放进一个具体场景** STM32F103采用Cortex-M3内核，实现ARMv7-M架构。你的C程序编译后由内核执行；程序配置ADC寄存器后，实际采样由芯片的ADC外设完成。内核不会因为会执行加法，就天然具备ADC、串口或GPIO。

**为什么面试会问** 同用M3的两款MCU，基本异常模型相近，但引脚、ADC通道、时钟树和DMA路由可以不同。因此内核问题查《权威指南》或PM0056；STM32外设问题查对应器件手册和RM0008。不能拿内核书代替整颗芯片手册。

**一句口述** 我先区分架构、内核和器件：ARMv7-M规定规则，Cortex-M3实现处理器，STM32F103在此基础上集成存储器与外设。

**检查自己是否懂了** 问：两块M3板能否直接共用全部初始化代码？答：不能。内核相关部分可能复用，外设地址、时钟、引脚和启动配置必须逐项核对。

## Q02 M3为什么是32位，却能执行16位指令？

A 必学 | 一题一页，先理解再复述

**先读指南** 第2章p24~26；第4章p50~51：看指令编码与Thumb状态。

**没懂再查** 第1章p15~19；第4章p57~58：看架构演进及不支持的指令。

**两种位宽说的不是一件事** 32位内核主要指通用寄存器和基本数据处理能力；16位或32位指令指的是一条机器指令占多少编码位。就像一个32位寄存器可以存一个很小的整数，处理器的数据能力不要求每条指令都用相同长度表达。

**M3实际怎样执行** M3使用Thumb执行状态，包含16位和32位的Thumb/Thumb-2编码，不执行传统A32 ARM指令。常见简单操作可能用较短编码，复杂操作或更大的操作数范围可能需要较长编码，最终由编译器和汇编器选择。

**为什么要这样设计** 短编码能减少Flash占用和取指流量，较长编码又能表达更多功能。不能据此认定16位指令一定更快，也不能把32位指令解释为一次执行两条16位指令；耗时还取决于操作、流水线和存储等待。

**名字不要混淆** ARM7、ARMv7与Cortex-M7分别属于不同命名层次。ARM7TDMI属于ARMv4T，而M3属于ARMv7-M；看到旧题写“ARM支持ARM态和Thumb态”，必须先问它讨论哪类处理器。

**口述与自测** 口述：数据位宽与指令编码宽度是两个概念。问：把M3返回地址最低位清零，能切入ARM态吗？答：不能，M3不支持该状态，错误入口可能触发UsageFault。

## Q03 流水线、哈佛结构和总线有什么联系？

B 补强 | 一题一页，先理解再复述

**先读指南** 第6章p99~104：先看流水线和总线接口。

**没懂再查** 第5章p83~85；第9章p148~149：再看地址映射与中断延迟。

**流水线解决什么** 把执行过程分成取指、译码、执行等阶段后，不同指令可以同时处在不同阶段。M3采用三级流水线。这主要提高连续指令的吞吐量，并不等于一条指令从开始到结束只需要一个周期。

**总线解决什么** M3具有分离的指令和数据访问通路，可让部分取指和数据访问重叠。内核对软件仍提供统一地址空间，因此“哈佛结构”不应被背成程序和数据永远无法相互访问。实际并行程度还取决于MCU内部总线和存储器连接。

**用中断入口理解** 异常入口既需要把寄存器写入栈，又需要读取向量和处理程序指令。不同通路可让这些工作部分并行，这也是中断响应较快的原因之一；但Flash等待或总线争用仍能拉长时间。

**联系DMA** CPU和DMA虽然不是同一个执行单元，但可能争用同一SRAM或总线路径。因此DMA减少了CPU逐次搬运的指令开销，不代表完全不占带宽，更不代表打开DMA后CPU性能绝不受影响。

**口述与自测** 口述：流水线提高吞吐，分离通路增加并行机会，最终速度仍受资源等待限制。  
问：72MHz能否推出每秒完成7200万次任意C运算？答：不能，C运算对应指令数量和每条指令耗时都不同。

## Q04 R0~R15、xPSR和函数调用怎样配合？

A 必学 | 一题一页，先理解再复述

**先读指南** 第3章p35~40：重点R13、R14、R15和xPSR。

**没懂再查** 第10章p151~153：调用约定；第9章p141~143：异常保存的寄存器。

**先记职责，不硬背表格** R0~R12主要承载运算数据；R13是SP栈指针；R14是LR，普通函数调用时保存返回信息；R15是PC，控制执行位置。xPSR包含运算标志APSR、当前异常号IPSR以及执行状态EPSR。

**一次函数调用发生什么** 常见AAPCS约定用R0~R3传递前部参数，返回标量结果通常用R0。需要保留的寄存器按调用约定由调用者或被调用者保存。参数含64位数、结构体时还涉及对齐与拆分，不能机械地说“前四个参数一定分别放R0~R3”。

**为什么需要栈** 寄存器数量有限，函数还可能有局部对象、额外参数和需要保留的返回地址。编译器会按需要使用栈，但优化后有些局部变量可只留在寄存器，甚至被消除，不能把每个C变量都对应为一个固定栈地址。

**调试时看什么** IPSR为0表示Thread模式；非0表示当前异常号。普通函数中的LR与异常入口写入LR的EXC\_RETURN含义不同，后者不是真正代码地址。区分这点才能理解HardFault现场。

**口述与自测** 问：C函数里的局部变量都在SRAM栈中吗？答：不一定，要看优化和实际生成代码。口述时先说明寄存器职责，再谈调用约定与栈，最后区别普通返回和异常返回。

## Q05 Thread、Handler与特权级是怎样切换的？

A 必学 | 一题一页，先理解再复述

**先读指南** 第3章p40~42：CONTROL、模式与权限变化。

**没懂再查** 第2章p24；第7章p124~126：SVC进入受控服务。

**先分开两个维度** Thread和Handler回答“正在执行普通程序还是异常处理”；特权和非特权回答“允许访问哪些受保护资源”。M3的Handler始终是特权级；Thread既能特权运行，也能非特权运行。不是四种等价的工作模式。

**复位后的起点** 复位后从特权Thread开始。特权代码可以通过CONTROL降低Thread权限；非特权程序不能自己随意写回特权状态，否则权限隔离就没有意义。需要经过受控的异常服务，例如SVC处理程序完成允许的系统服务。

**进入中断时** 不论普通程序原先是否为特权Thread，异常服务都在特权Handler中执行；正常异常返回后回到被打断的上下文。这里的自动模式变化，和应用程序主动调用一个普通C函数不是同一件事。

**与FreeRTOS的关系** 有RTOS并不意味着所有任务都是非特权。普通非MPU端口和带MPU保护的端口使用方式不同，必须看实际配置；不能把指南中的保护型OS示例当成你的每个工程都已经实现了权限隔离。

**口述与自测** 问：调用一个名字叫OS\_Service的C函数就进入Handler吗？答：不会，名字不决定CPU模式。口述：模式表示执行上下文，权限表示访问能力；异常入口能改变模式，普通函数调用不能。

## Q06 MSP和PSP怎样工作？异常时现场存哪？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第3章p36~37、p44~47：双栈与压栈示意。

**没懂再查** 第12章p179~181；第9章p141~147：双栈系统与异常返回。

**它们首先是两个指针** MSP和PSP指向栈所在的内存，并不是两块由内核自动分配好的RAM。软件必须安排栈区域、初始化指针并保证容量。复位使用MSP，Handler使用MSP；Thread可由CONTROL.SPSEL选择MSP或PSP。

**按时间顺序看一次中断** 假设任务正在Thread模式使用PSP。异常到来时，硬件先把任务的基本现场压入这个PSP栈；随后处理程序在Handler模式使用MSP。处理程序自己的调用和局部数据使用MSP，不能说任务现场也被搬到了MSP。

**返回时怎样接上** EXC\_RETURN指明返回模式与应使用的栈。若正常返回这个任务，硬件从PSP恢复之前保存的现场。如果RTOS换了任务，端口先选好新任务的PSP和额外上下文，再让硬件完成返回。

**双栈为什么有用** 把任务栈与异常使用的主栈分开，更容易管理各任务上下文和中断嵌套开销。但两个指针不等于自动防溢出；没有合适的边界检查或MPU配置时，栈仍可能写坏别的内存。

**口述与自测** 问：Thread是否一定用PSP？答：不是，裸机程序常一直使用MSP。必须能说清“从原栈保存现场，进入Handler后用MSP”，而不是笼统说“中断来了换栈”。

## Q07 满递减栈、32字节现场和8字节对齐怎么区分？

A 必学 | 一题一页，先理解再复述

**先读指南** 第3章p44~46；第12章p181~182。

**没懂再查** 第9章p141~143：基本异常栈帧；第10章p152~153：调用约定。

**满递减说的是增长方式** M3压入一个32位数据时，SP先减4，再把数据写到新位置；SP指向当前已存放的栈顶元素。弹出时先读，再调整SP。假设SP从0x20001000开始，压入一个字后变为0x20000FFC。

**32字节说的是基本异常现场** M3异常入口自动保存8个32位寄存器，共32字节。这不包括中断函数额外局部变量、编译器保存的其他寄存器，以及嵌套异常，所以不能把32字节当成任务栈或中断栈的足够容量。

**对齐是另外一个约束** AAPCS在公开调用边界要求SP满足8字节对齐。异常入口的STKALIGN机制可以按需要插入额外对齐空间，并在入栈xPSR中记录；返回时据此恢复。对齐填充不属于那8个寄存器的数据。

**为什么不随意改栈** 在普通C函数中直接切换SP，编译器可能还要按旧栈访问局部变量或返回信息，从而出错。双栈初始化与任务切换应使用经过验证的启动代码或端口，不照抄几句汇编到任意函数中。

**口述与自测** 问：嵌套两次中断只需64字节栈吗？答：那只是两个基本现场的部分开销，还要计入调用、局部变量、对齐等。栈大小需结合最坏调用链和运行测量。

## Q08 复位后CPU如何找到第一条指令？

A 必学 | 一题一页，先理解再复述

**先读指南** 第3章p47~48：复位序列及MSP/PC示例。

**没懂再查** 第3章p43~44；第10章p153~154：向量表与启动。

**先读数据，再执行代码** 复位后内核从复位向量表读取两个关键值：第0项作为初始MSP，第1项作为Reset\_Handler入口。它不是从地址0直接执行一条普通跳转指令，这一点与一些传统ARM启动资料不同。

**为什么先有栈** 异常可能很早发生，启动代码自己也可能调用函数；没有有效栈就无法安全保存现场。因此初始MSP应位于合法RAM中并满足对齐，通常指向分配给主栈区域的高地址端。

**为什么入口常是奇数** M3处理程序入口包含Thumb状态标记，最低位应为1；实际指令地址按相应对齐执行。这里说的是代码入口，不是向量表中的初始MSP也必须为奇数。指南相关概述不能过度泛化。

**放到STM32里** STM32通过启动映射机制让Flash、系统存储器或SRAM出现在启动所需的映射位置，具体受器件启动配置影响。M3给出复位规则，器件负责连接哪片存储器。

**检查与口述** 若程序到不了main，先看初始MSP是否合法，再看复位向量及Reset\_Handler是否正确，最后追初始化流程。问：烧录成功能证明复位入口正确吗？答：不能，链接地址、启动配置仍可能不匹配。

## Q09 向量表、异常号、IRQ号与VTOR怎样对应？

A 必学 | 一题一页，先理解再复述

**先读指南** 第3章p43~44；第7章p115~116：向量表定位。

**没懂再查** 第8章p135~136：初始化与重定位示例。

**向量表是地址表** 除首项初始MSP外，主要表项保存异常处理程序入口。外部IRQ0对应异常号16，因为前面为系统异常预留了位置；在M3中，异常号n对应表偏移 $4 \times n$ 字节。

**一个简单计算** 若外部IRQ5到来，则异常号为21，入口表项相对表基址偏移84字节，即0x54。实际工程不建议手工拼这些索引，应使用芯片启动文件与CMSIS中的IRQn定义。

**VTOR的作用** VTOR告诉内核异常向量表在哪里。把向量表迁移到RAM或应用程序起始位置时，需要让内容、地址和对齐都正确。它只改变查找入口的位置，不会自动初始化栈、清除外设状态或完成IAP跳转。

**两个易错点** 第一，NVIC管理外部中断不代表系统异常也都用同一EnableIRQ调用。第二，拷贝了一张表但没有修改VTOR，处理器可能仍查旧表；只改VTOR却没有有效表项同样会出错。

**口述与自测** 问：PendSV是IRQ14吗？答：不是，它是系统异常号14，不能与外部IRQ编号混用。口述先说表项内容，再解释系统异常与外部IRQ的编号转换，最后说明VTOR用于定位。

## Q10 Reset\_Handler到main，中间做了什么？

A 必学 | 一题一页，先理解再复述

**先读指南** 第10章p151～154；第19章p254～264：启动与工具链示例。

**没懂再查** 第3章p47～48；第10章p159～164：内存配置与启动相关示例。

**main不是硬件复位入口** Reset\_Handler负责把芯片和C运行环境带到可执行应用的状态。常见步骤包括系统初始化、复制有初值的可写数据、清零零初始化区域、必要的运行库初始化，之后才调用main。

**用变量理解初始化** int a=5需要在运行时的RAM中得到5；初始值一般随镜像保存在Flash，启动时复制过去。静态存储期的int b未显式初始化时应为0，这通常由启动代码清零.bss完成。不是RAM断电后天然全零。

**流程没有唯一固定顺序** SystemInit、C运行库入口和数据初始化的相对位置受工具链与启动模板影响。学习时应打开自己的startup文件，沿Reset\_Handler找调用链，而不是背某张图里永远先调用哪个函数。

**最小验证动作** 只读查看启动文件中的数据复制、清零循环及main调用；在调试器设置Reset\_Handler和main断点，验证执行经过哪里。不必为理解原理修改现有生产工程的启动代码。

**口述与自测** 问：全局变量初值错误，是否一定是业务代码写坏？答：也可能是链接布局或启动初始化不匹配。口述：硬件确定入口，启动代码建立C环境，应用再从main开始。

## Q11 如何用链接脚本与Map判断Flash/RAM占用？

A 必学 | 一题一页，先理解再复述

**先读指南** 第10章p159：数据区域；第19章p260～264：数据、链接脚本与C运行示例。

**没懂再查** 第3章p44～48：栈与启动；指南是旧工具链，具体语法以本机GCC工程为准。

**先认识几个段** 常见 `.text` 保存代码，`.rodata` 保存只读常量，`.data` 保存运行时可修改且有初始值的数据，`.bss` 保存零初始化对象。段名是常见约定，实际放在哪里由链接配置和启动代码共同决定。

**为什么.data会占两边** 程序掉电后仍需记住初始值，所以加载镜像通常在Flash；运行期间变量又需要可写，执行地址通常在RAM。复制后同一逻辑数据分别有加载位置和运行位置，不能只算其中一边。

**Map怎样读** 先找MEMORY区域的起址与长度，再看输出段大小、地址和符号；确认栈、堆或RTOS heap数组由哪种方式预留。有的栈/堆通过独立段保留，有的通过边界符号管理，不能只按 `.bss` 大小推断剩余RAM。

**为什么编译成功仍会溢出** 链接能检查静态区域是否装得下，却通常不知道运行期最深函数调用、递归和动态分配峰值。还要结合栈水位、溢出检查和heap最小剩余量；Map与运行测量是互补关系。

**口述与自测** 问：`.bss` 为2KB，是否要在Flash镜像中存2KB的零？答：通常不必，启动程序按范围清零。问：删一个大局部数组主要节省什么？答：可能降低栈需求，须看优化与实际代码。

## Q12 存储映射与Normal/Device属性为什么重要？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第5章p83~87：映射、属性与默认许可。

**没懂再查** 第14章p201~208：MPU区域属性；第4章p57：屏障指令入口。

**地址空间不是实物容量** M3的32位地址能描述4GB空间，内核将其划分为代码、SRAM、外设和系统区域。芯片可能只实现很小一部分，访问一个数字上存在但硬件未实现的地址并不安全。

**属性决定怎样访问** Normal适合普通存储器，在规则允许时可有缓冲、缓存或访问次序优化；Device适合带副作用的外设寄存器，不能随意当普通数组进行推测访问或合并。Strongly-ordered具有更严格的访问约束，但也不意味着所有同步问题自动消失。

**另一些属性是什么** 可执行决定能否从区域取指；访问权限区分特权/非特权及读写能力；可缓存、可缓冲和共享属性影响访问如何经过存储系统。M3标有可缓存属性，不代表具体芯片一定装有D-Cache。

**用寄存器举例** 读某个状态寄存器可能清标志，写1可能清零而不是置1。因此对外设执行memcpy或普通读改写前，必须确认访问宽度和副作用。volatile只影响编译器相关访问，不能把Device区域变成普通RAM。

**口述与自测** 问：地址能转成指针，是否就能安全读写？答：还须确认地址实现、权限、类型、对齐和寄存器语义。学习目标是解释这些约束，不是背4GB地图的每个空洞。

## Q13 大小端与非对齐访问，分别影响什么？

A 必学 | 一题一页，先理解再复述

**先读指南** 第5章p93~98：非对齐访问与大小端。

**没懂再查** 第4章p58~60、p68~69：加载宽度、符号扩展与字节反转。

**大小端决定字节排列** 把32位数0x12345678放入小端内存，从低地址到高地址依次为78、56、34、12。它只描述多字节对象的内存排列，不决定UART一字节内先发哪一位；通信协议的字节顺序还要另外约定。

**对齐决定起始地址条件** 32位字自然对齐通常要求地址可被4整除，16位半字要求可被2整除。M3支持部分普通加载/存储的非对齐访问，但可能拆成多次总线访问，存在性能代价；部分指令、内存类型或配置仍不允许。

**C语言还有自己的规则** 即使硬件某条LDR支持非对齐，也不等于任意强制转换指针都符合C的对齐和别名规则。解析串口字节流时，显式移位组合或按规则复制到正确类型对象，比随意把buf+1转为uint32\_t\*更稳妥。

**具体例子** 协议规定低字节先发，两个字节0x34、0x12可按无符号运算组合成0x1234。若协议规定高字节先发，同样两个字节解释就不同；不能把本机的小端排列当作对端协议的保证。

**口述与自测** 问：packed结构体能保证在所有ARM上直接访问都安全且高效吗？答：不能，还受编译器生成代码、地址对齐、访问指令和硬件限制影响。

## Q14 位带为什么能改单个位？能替代互斥锁吗？

B 补强 | 一题一页，先理解再复述

**先读指南** 第5章p87~93：位带区、别名区与例子。

**没懂再查** 第5章p95~97：对比独占访问；第3章p39~40：对比中断屏蔽。

**问题来自读改写** 普通  $x \mid= \text{mask}$  可能先读整个字、在寄存器里修改、再写回。若ISR在中间也改了同一个字，后写入的旧副本可能覆盖另一个上下文的修改。单独一条C语句不等于硬件原子操作。

**位带提供地址映射** 在支持的位带区域里，每一位对应别名区域中的一个字地址。访问这个别名地址，硬件对原地址的目标位操作，不用软件先读整个字再写回。先确认目标芯片及寄存器支持，不能推广到全部Cortex-M。

**算一次就理解** SRAM位带基址0x20000000，别名基址0x22000000。原地址0x20000004的bit3，对应  $0x22000000 + 4 \times 32 + 3 \times 4 = 0x2200008C$ 。这里4是字节偏移，不是第4个32位字。

**它的边界** 原子地写一个位，不等于“检查空闲再占用”这两个动作合起来原子，也不等于一个结构体更新完整。RTOS互斥锁还管理所有者、等待队列和优先级继承，位带不能直接替代它。

**口述与自测** 问：用位带先读锁为0再写1，两个任务一定不会同时拿锁吗？答：不能保证，两次访问之间仍有竞争窗口。先说明单次位访问原子，再说明多步协议仍须同步。

## Q15 LDREX/STREX如何避免读改写丢失？

B 补强 | 一题一页，先理解再复述

**先读指南** 第5章p95~97：互斥/独占访问机制与示例。

**没懂再查** 第4章p56：指令表；官方02的LDREX、STREX条目核对限制。

**先区分名字** 指南中的“互斥访问”主要指exclusive access指令机制，不是FreeRTOS互斥锁对象。LDREX读取数据并建立独占监视状态；STREX尝试在条件仍满足时写回，同时返回成功或失败的状态。

**按流程理解** 读取旧值 → 在寄存器计算新值 → 条件写回。若STREX返回0，写入成功；非0则这次未成功写入，算法通常重新读取再算。它避免把已经失效的旧计算结果无条件覆盖回共享位置。

**为什么需要重试** 异常或其他规定的情况可能破坏独占条件。不能只执行一次STREX后忽略状态，也不能在两条指令之间塞入很长的无关操作。地址、宽度、对齐、内存类型必须符合架构及器件规则。

**不能保证什么** 对DMA、其他总线主设备的独占监视支持必须查具体实现，不可当成所有访问者都会自动配合。M0/M0+不能照搬M3这组指令；应用任务需要保护I2C事务时通常仍优先使用RTOS互斥锁。

**口述与自测** 问：STREX失败，能否说共享资源已被锁住且本任务持锁？答：不能，只表示本次条件写没有成功。口述重点是“读、计算、尝试写、失败重试”，不是“硬件自动加一把万能锁”。

## Q16 MPU与MMU有什么区别？MPU能防止什么？

B 补强 | 一题一页，先理解再复述

**先读指南** 第14章p201~208：区域、权限与使能。

**没懂再查** 第5章p85~87；第3章p40~42：属性与特权；具体器件是否实现MPU须另查。

**MPU按区域检查访问** 软件为若干地址区域设置读写权限、可执行性及属性，CPU访问时进行检查。不符合规则的访问可触发MemManage。它不是在访问出错后替你修复指针，而是尽早阻止或暴露部分非法访问。

**与MMU的区别** 典型MMU还负责虚拟地址到物理地址的转换，支持更复杂的地址空间管理。M3的MPU不提供这套虚拟地址转换；不能把M3保护区域称为Linux式虚拟内存。

**用任务栈理解** 若硬件与软件支持，可在栈旁布置不可访问区域，越界触及该区域时产生Fault。但越界若落入另一合法区域，或保护配置不合适，仍可能漏掉；MPU并不替代边界检查和栈容量设计。

**阅读原书要纠错** 指南p206的XN表出现“2=允许取指”，而该位只有0/1；p207说明也有方向笔误。正确是XN=1禁止取指、XN=0不因XN而禁止取指，仍须满足其他执行条件。查官方03核对，不能照抄表中错误。

**口述与自测** 问：用了FreeRTOS就自动有MPU隔离吗？答：不是，必须芯片实现、端口支持且实际配置启用。M3内核支持选配功能不代表每款M3 MCU都带有它。

## Q17 异常、中断、Fault分别是什么关系？

A 必学 | 一题一页，先理解再复述

**先读指南** 第7章p109~110、p119~123。

**没懂再查** 第3章p42~44：异常类型与向量；附录E p315~322：定位。

**异常是总称** 在Cortex-M语境中，异常机制包含外部IRQ以及系统异常。外部IRQ可由芯片外设提出；SVC由指令触发；SysTick来自内核定时器；Fault用于处理执行或访问错误。不能把异常仅理解为程序出错。

**需要掌握的编号** Reset=1, NMI=2, HardFault=3, MemManage=4, BusFault=5, UsageFault=6, SVC=11, DebugMonitor=12, PendSV=14, SysTick=15。外部IRQ从异常号16开始；13是保留号。

**Fault不是同一类原因** MPU权限问题可能产生MemManage；总线返回访问错误可能产生BusFault；非法指令或非法执行状态可能产生UsageFault。可配置Fault未启用或不能正常处理时可能升级HardFault，因此进入HardFault还要查更细的状态。

**不要只背异常表** 若函数指针错误、栈损坏或访问无效地址，最终落入哪类Fault取决于实际执行路径及配置。CPU不会直接告诉你“第几行C代码的变量错了”，需要把状态位、栈现场和反汇编对应起来。

**口述与自测** 问：HardFault出现是否意味着硬件坏了？答：不能，软件越界、错误返回、权限和配置问题都可能触发。先分类、再取证，不凭处理函数名称猜原因。

## Q18 NVIC优先级数值越小，为什么越紧急？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第7章p110~114：有效位、固定优先级与分组。

**没懂再查** 第8章p131~134；官方01的NVIC\_SetPriority：核对库参数编码。

**这是架构约定** M3的可配置异常优先级数值越小越紧急。Reset、NMI和HardFault有固定的更高优先级，不参加普通IRQ的同样数值配置。FreeRTOS任务优先级往往是数值越大越高，两个系统不能混在一起比较。

**有效位不一定有8个** 优先级字段占一个字节，但芯片可能只实现高几位。假设实现4位，逻辑优先级0~15在寄存器中对应0x00、0x10等编码；低4位不起作用。实际有效位看芯片头文件和手册。

**为什么容易配置错** CMSIS NVIC\_SetPriority通常接收未移位的逻辑值，函数内部负责放到高位；BASEPRI等直接寄存器值则使用相应编码。把已经左移的数再交给会移位的API，会得到错误结果。

**用例子判断** 在全部有效位都用于抢占优先级的前提下，IRQ A逻辑值2比B的5更紧急。A若在B执行中到来且未被屏蔽，可以抢占B；不能据此认为任务优先级2也比任务5高。

**口述与自测** 问：知道寄存器8位就能设置256级有效优先级吗？答：不能，受芯片实现位数限制。回答时务必先说内核/芯片与分组前提。

## Q19 抢占优先级与子优先级怎样决定执行顺序？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第7章p112~115：PRIGROUP与抢占规则。

**没懂再查** 第9章p144~146：嵌套、尾链和晚到；第8章p135~136：配置示例。

**它们回答不同问题** 抢占优先级决定一个异常能否打断正在执行的另一个异常；子优先级在相同抢占级的多个请求都待处理时帮助选择先处理谁。子优先级更高，不能单独让它打断同抢占级ISR。

**例子一：正在运行** 设A和B抢占级都是2，A子级0、B子级1。B已经在ISR里执行时，A到来并不会仅凭子级更高就打断B；A先保持Pending，等到合适的异常边界再被处理。

**例子二：都在等待** 若A和B都Pending，且当前能响应它们，那么同抢占级下先选择子级更高者。若可比较的优先级完全相同，架构还有固定仲裁规则；这不是随机轮转，更不是FreeRTOS的时间片。

**分组不是额外增加位数** 有限的有效优先级位被分给抢占级与子级。给子级更多位就会减少可用抢占层次。常见FreeRTOS Cortex-M配置倾向将有效位用于抢占，具体按所用端口要求核对。

**口述与自测** 问：两个同抢占级中断能否互相嵌套？答：不能仅靠子优先级实现。先判断能否抢占，再判断等待者的选择顺序，不把两步倒过来。

## Q20 Enable、Pending、Active为何必须分开看？

A 必学 | 一题一页，先理解再复述

**先读指南** 第7章p116～119；第8章p130～133。

**没懂再查** 第8章p135～136：初始化过程；官方01的EnableIRQ与ClearPendingIRQ。

**三个词对应三个状态** Enable表示允许这一路响应；Pending表示请求已经待处理；Active表示这路异常正在被服务。关闭Enable不等于请求从未发生，也不等于把Pending或外设状态清掉。

**从外设走到CPU** 外设先产生事件并设置自己的状态，符合其使能条件后提出中断请求；NVIC再结合使能、优先级、屏蔽与当前执行状态决定是否进入ISR。任何一层阻断，都可能出现“外设有标志但ISR没进”。

**返回后为什么反复进** ISR若没有按外设手册清除或处理请求源，中断条件可能仍持续有效，返回后再次Pending。只清NVIC位而忽略外设来源，通常不能根除问题；清标志的具体写法还可能是写1清零。

**诊断顺序** 先查事件是否发生及外设标志，再查外设中断开关、NVIC使能/挂起、PRIMASK/BASEPRI、优先级和向量入口。调试器连续读取某些寄存器还可能有读清零副作用，观察也需谨慎。

**口述与自测** 问：禁用EXTI对应IRQ后再使能，旧请求会自动消失吗？答：不会保证，需分别核对外设挂起与NVIC状态，并按业务决定是否丢弃旧事件。

## Q21 PRIMASK、BASEPRI、FAULTMASK怎样屏蔽？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第3章p39~41；第8章p133~134。

**没懂再查** 第7章p110~115：优先级基础；官方02的三个屏蔽寄存器。

**PRIMASK是较粗的开关** PRIMASK=1阻止可配置优先级异常被响应，但NMI和HardFault仍不受它屏蔽。常见\_\_disable\_irq使用这一机制。它不停止外设计数、DMA搬运，也不保证新的请求不进入Pending。

**BASEPRI是阈值** BASEPRI非零时阻止达到该阈值或更低紧急程度的可配置异常，高于阈值的紧急中断仍有机会响应；0表示不使用这道屏蔽。配置时必须采用有效优先级位对应的编码，不直接混用任务优先级。

**FAULTMASK范围更广** FAULTMASK可以连HardFault一起屏蔽，但不能屏蔽NMI，且有特殊清除规则。它不应被当作普通应用同步首选。屏蔽Fault也不表示导致Fault的错误已经修复。

**为什么RTOS常用阈值** 端口可以暂时阻止会访问内核数据的中断，同时保留特别紧急且不调用内核API的中断。这要求ISR优先级与FromISR门槛配置一致。M0/M0+没有BASEPRI，不能照抄M3做法。

**口述与自测** 问：BASEPRI置0是关闭所有中断吗？答：恰好不是，是解除BASEPRI屏蔽。问：关中断后DMA数组不变吗？答：不保证，因为DMA是独立硬件访问者。

## Q22 异常进入、自动入栈与EXC\_RETURN如何连起来？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第9章p141～147：入口、栈布局、返回编码。

**没懂再查** 第3章p44～48；第12章p181～182：栈与对齐；附录E p316～319。

**入口必须保留被打断者** M3基本异常现场包括R0～R3、R12、被打断时的LR、返回PC和xPSR。它们压到当时使用的栈，随后处理程序进入Handler并使用MSP。编译器和RTOS还可能另外保存其他寄存器。

**两个LR不要混** 栈里保存的是被打断代码原来的LR；进入异常后当前LR被写成EXC\_RETURN。前者属于旧现场，后者告诉硬件这次异常退出时应返回哪种模式、从哪个栈恢复。

**三个常见编码** M3的0xFFFFFFF1表示返回Handler并用MSP；0xFFFFFFF9表示返回Thread并用MSP；0xFFFFFFF9表示返回Thread并用PSP。它们不是普通函数地址，也不应在普通C代码中随便调用。

**恢复链** 处理程序结束 → 通过合法异常返回触发硬件退出 → 按指定栈恢复基本现场 → 回到原执行位置或RTOS选择的新任务。若栈、xPSR或返回编码被破坏，异常退出自身也可能Fault。

**口述与自测** 问：LR等于0xFFFFFFF9，说明程序跑飞到高地址吗？答：若正处在M3异常入口上下文，它可能是正常返回控制值。必须结合上下文，不能只凭数值判断。

## Q23 尾链、晚到与普通中断嵌套有何不同？

B 补强 | 一题一页，先理解再复述

**先读指南** 第9章p144~146：三种情形的图。

**没懂再查** 第7章p112~119：优先级及Pending；第9章p148：响应时间前提。

**嵌套：处理程序已经运行** 低优先级ISR执行到中途，更紧急且允许响应的中断到来。硬件再次保存当前ISR现场，再运行高优先级ISR；高优先级结束后才有机会回到被打断的低优先级ISR。嵌套增加栈开销。

**尾链：一个结束，另一个在等** 当前ISR即将退出时，存在合适的待处理异常。内核可以省去先恢复普通上下文、立刻又把同一上下文压回去的重复过程，直接转到下一ISR。它优化的是异常衔接，不是调度器的同优先级轮转。

**晚到：入口还没准备完** 低优先级异常正在入口入栈阶段，更紧急异常及时到来，内核可以让已开始保存的现场为更紧急异常服务，先执行它。原低优先级请求并不是因此丢失，而是等待后续合适时机。

**怎样记得住** 已经执行ISR再被打断，想嵌套；ISR退出时发现有人等，想尾链；入口阶段被更紧急请求改变先后，想晚到。判断的是发生时刻，而不只是优先级高低。

**口述与自测** 问：尾链会跳过对原程序现场的最终恢复吗？答：不会。它省去中间多余的恢复/重存，最后没有更合适异常时仍需要按规则返回。

## Q24 中断延迟为什么不能只算12个周期？

A 必学 | 一题一页，先理解再复述

**先读指南** 第9章p148~150：延迟与异常期间错误。

**没懂再查** 第6章p99~104；第15章p219~226；第16章p230~231：调试影响与DWT周期计数。

**先定义起点与终点** 中断延迟通常讨论请求被处理器识别后到ISR第一条指令执行的时间；外部物理事件到业务完成还包含输入同步、外设检测、软件处理和后续任务等待。因此测量前必须定义自己到底测哪一段。

**12周期有条件** M3常见的12周期入口指标依赖无额外等待、可以立即响应及合适总线条件等。若中断被屏蔽、已有更高紧急度ISR运行，或者取向量/入栈访问等待，真实延迟就会增加。

**用GPIO脉冲测什么** 在ISR入口置位诊断GPIO、出口清零，可测近似执行时长；若同时捕获外部事件边沿到GPIO置位，可估计端到端入口延迟。GPIO操作本身也有耗时，测量结果要注明起终点和仪器分辨率。

**软件侧还能看什么** DWT周期计数器在支持并正确启用时可用于测周期，但需处理回绕、调试停机与测量代码开销。断点会改变时序，不能用逐步执行时间推算真实运行延迟。

**口述与自测** 问：缩短ISR能否降低别的中断等待？答：可能，特别是被它挡住的较低紧急度请求；但外设同步和其他屏蔽来源仍需单独分析。

## Q25 SysTick、主频与RTOS Tick如何计算和区别？

A 必学重点 | 一题一页，先理解再复述

先读指南 第8章p137~139；第13章p189~191。

没懂再查 第12章p179~182：OS衔接；FreeRTOS具体行为查官方04/05，指南不是API手册。

**主频是CPU节拍，Tick是软件时基** SysTick是内核中的24位递减计数器。选择输入时钟并设置LOAD后，可以周期性产生异常；RTOS常借它推进Tick计数，但也允许采用其他时基实现。它不是只能服务RTOS。

**算一个完整例子** 周期 $T = (LOAD + 1) / f_{\text{输入}}$ 。若SysTick输入为72MHz、目标100Hz，则 $LOAD = 720000 - 1 = 719999$ ，一个Tick为10ms。不能写成71999999，否则既算成1秒又超出24位范围。输入究竟是HCLK还是其他来源必须查器件配置。

**Tick与时间片的关系** 在开启抢占与时间片配置时，Tick可促使同优先级就绪任务轮转；但高优先级任务在Tick中途被唤醒也可能立即抢占。当前任务阻塞时，调度器不必等它用满一个Tick才切换。

**延时精度的边界** 请求延时若干Tick，是按离散Tick计数决定何时变Ready，不保证恰好经过对应毫秒就开始执行；调用处于Tick内的相位和更高优先级任务都会影响实际墙钟时间。

**口述与自测** 问：100Hz Tick是否意味着所有任务每10ms才允许切换一次？答：不是。先算Tick长度，再分清时基、时间片、阻塞唤醒与抢占。

## Q26 SysTick、SVC、PendSV怎样完成任务切换？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 第7章p124~127；第12章p179~182。

**没懂再查** 第9章p141~147；官方04的xPortPendSVHandler、vPortSVCHandler、xPortSysTickHandler。

**三者分工** 常见FreeRTOS M3端口用SysTick推进时间并判断是否需要调度；PendSV承担上下文切换；SVC用于启动首任务等端口行为。不是每次进入一个内核C函数都产生SVC，也不是每次Tick都必然换任务。

**换出任务A** 异常入口先由硬件保存A的基本现场；PendSV端口读取当前PSP，再用软件保存R4~R11等额外上下文，把新的栈顶记录到A的任务控制块。这样任务下次恢复时，寄存器状态才完整。

**换入任务B** 调度器选择就绪任务B → 端口从B的任务控制块取得保存的栈顶 → 恢复软件保存的寄存器并设置PSP → 异常返回由硬件恢复B的基本栈帧。B继续执行，好像只是暂停过一段时间。

**为什么PendSV很低** 让切换推迟到合适的异常边界，避免在尚有需要完成的高优先级ISR链中处理任务现场。应使用端口要求的优先级配置，而不是随意把PendSV调高来追求“更快”。

**口述与自测** 问：硬件既保存8个寄存器，为何RTOS还保存R4~R11？答：完整任务上下文多于基本异常帧。指南提供机制；具体额外字段、浮点现场和首任务路径必须看对应端口。

## Q27 FromISR与xHigherPriorityTaskWoken到底管什么？

A 必学 | 一题一页，先理解再复述

**先读指南** 指南第7章p124~127、第9章p141~144：只作为异常与切换的原理背景。

**没懂再查** API本身不在本指南中。直接查官方05的中断管理章节；M3切换实现查04及对应FreeRTOS版本。

**为什么分两套API** 普通任务API可能让调用任务阻塞等待，而ISR不是可以挂进任务等待队列的普通任务。FromISR版本采用适合中断上下文的操作方式，不提供在ISR内等待资源的普通阻塞语义。

**标志表示什么** 调用前通常将xHigherPriorityTaskWoken初始化为pdFALSE。相关ISR API若使比被中断任务优先级更高的任务就绪，会按约定设置该标志；随后portYIELD\_FROM\_ISR等端口宏请求在合适的异常边界切换。

**一个对比** 任务A优先级1运行，ISR唤醒优先级2的B，常见情形下标志变true；若唤醒的B也是1，不能说该标志必为true。之后谁运行仍取决于调度配置和就绪状态。

**还有优先级门槛** 并不是名字带FromISR就可在任意IRQ优先级调用。在使用BASEPRI的端口中，过于紧急、不会被内核临界区屏蔽的ISR，不能随意访问内核API，否则可能打断内核共享数据更新。

**口述与自测** 问：标志true表示中断里面已经完整执行了被唤醒任务吗？答：不是，它报告切换需求，实际执行还要经过端口与异常退出。学习到能解释此链即可，不需背每条汇编。

## Q28 临界区为什么要恢复旧状态，而不是直接开中断？

A 必学 | 一题一页，先理解再复述

**先读指南** 第3章p39~41；第8章p133~134：屏蔽寄存器。

**没懂再查** 第5章p91~97：共享访问；RTOS嵌套实现查官方04的临界区函数。

**临界区要保护一段不可交叉的操作** 若任务和ISR同时读改写共享数据，临界区可以暂时阻止指定范围的中断插入。它保护的是短操作的执行条件，不应把耗时I2C通信、显示整帧或等待外部事件都塞进去。

**嵌套时会发生什么** 外层已经关中断，调用的内部函数又关一次；若内层结束时无条件开中断，外层尚未完成的共享更新就暴露出来。正确做法是保存进入前状态，退出时恢复，或由RTOS规定的嵌套机制管理。

**RTOS工程怎样处理** 任务上下文使用对应任务临界区API，ISR使用端口允许的ISR临界区API并保存必要状态。不自行混搭裸机开关中断、调度器暂停和RTOS临界区，三者保护范围不同。

**两个不能混用的方案** 互斥锁可让任务在资源不可用时阻塞并管理所有者；临界区通常保持很短且不阻塞。暂停调度器并不必然屏蔽中断，因此不能自动保护任务与ISR之间的竞争。

**口述与自测** 问：临界区内等DMA完成中断再退出可以吗？答：若把该中断屏蔽了就会永等，即使未屏蔽也可能严重损害实时性。应使用完成通知与任务等待，而非长时间关中断。

## Q29 volatile、原子性与数据一致性为什么不是一回事？

A 必学 | 一题一页，先理解再复述

**先读指南** 指南第5章p91~97：读改写竞争的硬件背景。

**没懂再查** 第4章p58~61：加载/存储；volatile完整语义属于C语言，指南不能替代C标准或编译器文档。

**volatile解决的是访问被优化的问题** 它用于需要保留可观察访问的对象，例如按规则访问外设寄存器。不能把它理解为任意代码都绕过缓存，或让编译器自动建立多任务同步协议；它也不是C语言中的原子类型。

**自增为什么会丢失**  $x++$ 可能是加载 $x$ 、加1、写回。若主程序读到10后被ISR打断，ISR把 $x$ 改成11；主程序回来仍把自己算的11写回，最终少了一次加1。给 $x$ 加volatile并不能合并这三个动作。

**单字完整不等于整帧完整** 即使对齐的单次访问具有相应原子性，读取温度、湿度两个字段时，仍可能读到一次更新前的温度与更新后的湿度。数据一致性是多个访问的共同约束，比单个字不撕裂要求更多。

**选择正确工具** 单生产者/消费者可设计所有权和发布协议；多任务资源可用互斥锁；很短的任务/ISR共享操作可按端口用临界区；硬件缓冲区还要考虑DMA。不能只凭“都加volatile了”结束评审。

**口述与自测** 问：双缓冲数组都加volatile后能保证LCD不读到写到一半的数据吗？答：不能，还需FREE/WRITING/READY/DISPLAYING等状态或等效所有权协议。

## Q30 DMB、DSB、ISB分别解决什么问题？

B 补强 | 一题一页，先理解再复述

**先读指南** 第4章p72~73：屏障专节；p56~57：指令表。

**没懂再查** 第5章p85~86：访问属性；原书p72将DMB写得像完成屏障、ISB写成最严格，须结合官方02纠正。

**先理解为什么需要屏障** 程序语句的书写顺序，不一定足以表达硬件所需的访问顺序和完成条件。写缓冲、不同访问路径或处理器执行环境变化，都可能要求软件在规定位置明确建立顺序。

**DMB：约束访问顺序** 它保证规定范围内前后显式内存访问满足所需顺序，但不要把它背成等待全部访问真正完成。重点是内存访问之间的顺序关系，不是让另一任务立刻运行。

**DSB：还等待相关操作完成** 它比单纯建立数据访问顺序多了完成方面的要求，常用于必须确保某些先前操作完成之后才能继续的场景。它不能把一个本来非法的地址访问变成合法，也不能替代设备状态轮询。

**ISB：更新后续指令执行环境** 它使后续指令按更新后的执行环境重新取指处理，常与修改某些系统控制状态有关。它并非“最强的数据屏障”，因此不能将DMB、DSB、ISB仅排成弱、中、强并随意互换。

**口述与自测** 问：DMB能保证LCD任务已用完缓冲区吗？答：不能，所有权和完成通知不是屏障负责的事。应用层一般遵循CMSIS/厂商库和端口规定使用，不到处盲目插屏障。

## Q31 DMA为什么会共享缓冲区出问题？

A 必学；Cache部分B补强 | 一题一页，先理解再复述

**先读指南** 指南第6章p103~104；第5章p85~86：总线与访问属性背景。

**没懂再查** 指南不提供M7缓存/DMA完整教程；查官方07的Cache维护资料及具体MCU的DMA章节。

**两条独立执行线** CPU执行程序，DMA按外设请求或配置搬运数据。任务被阻塞、CPU关中断或进入某些等待，不等于DMA自动停止。因此CPU端临界区不能单独保证正在被DMA写入的数组不变。

**最常见的半新半旧** DMA循环写入一帧数组，LCD或计算任务同时从头读取。当两者速度和时间重叠，读取者可能前半段拿到旧帧、后半段拿到新帧，即使每个元素都是完整数字，整帧仍不一致。

**双缓冲需要交接** 生产者只写空闲缓冲，写完才发布就绪；消费者只读已完成缓冲，读完归还。若两块都被占用，应定义丢帧、增加缓冲或延后采集策略。事件通知告诉你有事发生，不自动管理具体是哪块数组。

**M7另有缓存问题** 带D-Cache时，CPU看到的缓存副本与DMA访问的RAM可能不同。需要按芯片指南进行发送前Clean、接收适当阶段Invalidate及缓存行对齐管理；盲目失效可能丢掉同一缓存行里别的脏数据。

**口述与自测** 问：DMA计数归零就等于屏幕已显示完成吗？答：不等于，它只证明对应搬运阶段结束，SPI线上的最后一位和LCD接收还属于后续层次，须看完整驱动完成语义。

## Q32 WFI/WFE与Sleep、Stop、Standby怎样联系？

A 必学 | 一题一页，先理解再复述

**先读指南** 第13章p191~193：睡眠/深睡眠及唤醒。

**没懂再查** 第4章p56~57；STM32具体Stop/RTC/时钟恢复不在本指南，查官方08的PWR、RCC、RTC章节。

**内核给出睡眠入口** WFI使内核等待符合条件的中断唤醒；WFE使用事件机制，已有事件可能让它立即返回。SLEEPDEEP向芯片侧表达深睡意图，SLEEPONEXIT影响异常返回后的休眠。执行一条指令不等于整块板都断电。

**芯片决定关哪些资源** STM32的Sleep、Stop、Standby由器件电源和时钟设计实现，保留内容、可用唤醒源和醒后路径不同。必须查具体型号，不能从M3内核支持深睡直接推断RTC、SRAM、PLL都如何变化。

**完整低功耗链** 完成必要工作 → 确认通信/缓冲区状态 → 设置唤醒源与期限 → 防止检查与入睡之间丢事件 → 进入模式 → 唤醒 → 按器件要求恢复时钟和外设 → 再运行应用。每一步都应有失败或超时策略。

**为什么醒后串口乱码** 若醒后系统时钟未恢复，而串口分频仍按旧频率理解，实际波特率可能变化。该现象应检查时钟来源、PLL状态与外设时钟，而不是一开始就怀疑蓝牙或串口模块损坏。

**口述与自测** 问：WFI返回就证明执行过预期唤醒ISR吗？答：不能一概而论，唤醒与异常响应的条件要分开核对。问：CPU睡了为何电流高？答：外部模块、稳压器、LED和调试连接仍可能耗电。

## Q33 RTOS Tickless与裸机RTC定时唤醒一样吗？

B 补强 | 一题一页，先理解再复述

**先读指南** 指南第13章p189~193：时基和低功耗的共同背景。

**没懂再查** 指南未系统讲FreeRTOS Tickless。查官方04的vPortSuppressTicksAndSleep及05的低功耗相关说明。

**共同目标** 两者都希望CPU在没有必要工作时休眠，避免无意义地持续运行。裸机可以由应用自己决定下一次采样时间；RTOS还必须兼顾所有任务等待期限和内核Tick计数。

**Tickless多做了什么** 当空闲条件满足时，系统预测允许休眠的时长，抑制不必要的周期Tick，并在醒后补偿经过的内核时间。期间若外部事件提前唤醒，也要按实际经过时间恢复，而不是总认为睡满了计划时间。

**不能直接关掉SysTick** 任务可能正在等待延时、队列或软件定时器期限。如果停了Tick却没有补偿和唤醒安排，任务超时管理会失真。端口还要处理计时范围、舍入误差及进入休眠前的竞争窗口。

**项目表述边界** 裸机用RTC每隔一段时间唤醒采集，应称RTC周期唤醒或低功耗调度；不能因为都用了睡眠就声称实现了FreeRTOS Tickless。是否启用需看对应端口和configUSE\_TICKLESS\_IDLE配置。

**口述与自测** 问：某任务1ms后应超时，能否无条件睡5秒？答：不能，最近的任务期限可能限制允许休眠时间。核心区别是RTOS需要维护全系统时间语义，而不只是单一应用闹钟。

## Q34 HardFault发生后，如何拿到可信现场？

A 必学重点 | 一题一页，先理解再复述

**先读指南** 附录E p315~319：栈定位及现场提取；第7章p119~123。

**没懂再查** 第9章p141~147；附录E p320~322：故障分类。原书p316的PendSV示例不能占用RTOS的PendSV。

**先保现场，不急着复位** 记录异常入口的EXC\_RETURN，再确定基本栈帧在MSP还是PSP；同时检查栈指针是否落在合法范围。若进入普通C处理函数后才随便取当前MSP，可能读到的是处理函数自己的栈，而不是最初现场。

**哪些数据最有用** M3基本帧从帧基址起依次包含R0~R3、R12、原LR、PC、xPSR，因此在有效基本帧中PC位于第6个字偏移，即24字节处。再看CFSR、HFSR；只有地址有效标志成立时才依赖MMFAR/BFAR。

**怎样缩小根因** 用Fault状态判断非法访问、状态或指令问题，再把入栈PC映射到ELF反汇编和源码，检查访问地址、指针生命周期、栈边界。非精确BusFault的入栈PC可能不是实际出错指令；损坏的栈帧也可能不可信。

**不要扩大故障** 在Fault里大量printf、动态分配或等待任务可能让问题更严重。应保留最小可靠记录，按产品策略停机或复位；日志若要跨复位保留，还需专门存储与初始化设计。

**口述与自测** 问：找到PC所在C行就能断定这一行算法错了吗？答：不能，该行可能只是使用了更早被写坏的指针；还要追查破坏来源。资料中的步骤是方法，不代表你已经完成实操验证。

## Q35 断点、观察点、寄存器和仪器分别怎样用？

A 必学方法 | 一题一页，先理解再复述

**先读指南** 第15章p219～226：调试接口与停机控制。

**没懂再查** 第16章p229～235：FPB/DWT组件；附录E p315～322：现场定位。

**断点：程序走到哪** 在关键调用处暂停，检查参数、返回值和调用栈，适合确认初始化或回调是否执行。它会改变运行节奏，所以一次断点下未复现竞争问题，不能证明实时并发没有缺陷。

**观察点：谁改了它** 硬件支持并配置合适时，可在指定地址被访问时停下，适合追踪某变量或缓冲区被谁改坏。观察资源数量有限，访问大小与触发条件受实现约束；对DMA写入能否触发必须查具体调试路径，不能保证。

**寄存器：硬件处在哪一层** 例如定时器CNT变化只说明计数器在动，不能证明TRGO配置正确；DMA计数递减说明有搬运进展，不能证明外部引脚波形符合协议。每个观测量只支持相应层次的结论。

**外部仪器：引脚实际发生什么** 逻辑分析仪观察数字时序和协议，示波器看模拟幅度与边沿，万用表适合静态电压等。SWD/JTAG是调试访问接口，UART日志是应用输出，两者不能互相替代。

**口述与自测** 问：SPI菜单不刷新，直接在每处加断点是否最佳？答：先用断点定位调用链；若怀疑时序，再使用诊断GPIO或非侵入记录。先说观测目标，再选工具，并明确它不能证明什么。

## Q36 LDR、STR和LDR =地址，怎样读懂？

A 必学指令 | 一题一页，先理解再复述

**先读指南** 第4章p49~50、p58~62：语法、加载存储和伪指令。

**没懂再查** 第5章p93~98：对齐与大小端；第19章p254~264：GCC语法背景。

**方括号表示访问内存** LDR R0, [R1]表示读取R1所指地址的一个字到R0；STR R0, [R1]表示把R0的字写到该地址。R1是地址，R0是数据，不要把加载地址本身和读取地址里的内容混成一步。

**等号形式是伪指令** LDR R1, =0x20000000意图把这个常数放入R1，汇编器可用常量池加载或其他合适编码实现；它不是读取地址0x20000000处的数据。接着LDR R0, [R1]才真正从该地址取数。

**宽度有区别** LDRB读字节并零扩展，LDRH读半字并零扩展，LDRSB/LDRSH进行符号扩展；STRB/STRH只写寄存器的低字节/低半字。外设允许的访问宽度必须按手册，不能只看自己想读取多少位。

**偏移与写回** [R1, #4]从R1+4访问但不因此修改R1；[R1, #4]!为相应预索引写回形式，会更新基址。读端口代码中的!时要留意指针也在变化，别误以为永远操作同一地址。

**口述与自测** 假设0x20000000处存5，执行LDR R1, =0x20000000；LDR R0, [R1]。问R0是多少？答5，R1仍为地址。无需先背全部指令，先把地址、数据、宽度、偏移分清。

## Q37 PUSH/POP、BL/BX怎样支撑函数调用？

A 必学指令 | 一题一页，先理解再复述

**先读指南** 第3章p37~38；第4章p53~54、p60~61：分支与栈操作。

**没懂再查** 第10章p152~153：AAPCS；第9章p143~147：区别异常返回。

**BL记录回来的地方** BL跳转到子程序并设置LR作为返回信息。普通叶子函数在未破坏LR时可用BX LR返回；如果函数还要调用其他函数，新的BL会覆盖LR，因此旧返回信息通常需要保存。

**PUSH/POP保存和恢复** PUSH {R4, LR} 可把需要保留的寄存器和返回信息压入栈；对应出口可用POP {R4, PC} 直接恢复R4并返回。这里是示意，真实函数保存哪些寄存器取决于调用约定、编译器与函数行为。

**为什么不能只说局部变量压栈** 栈还保存返回信息、被调用者保存寄存器、额外参数等内容。反汇编中出现SUB SP, #... 可能是在为局部区域留空间，不一定每一个栈字都能对应一个显式C变量。

**BX不保证切到ARM态** 在支持多状态的旧内核中分支状态规则更复杂；M3只支持Thumb，不能把BX当作任意切换ARM/Thumb的工具。在Handler中使用合法EXC\_RETURN还可触发异常返回，这与普通LR代码地址又不同。

**口述与自测** 问：函数A调用B，B又调用C，若B未正确保护需要保留的返回地址会怎样？答：可能无法回到A，出现错误返回或Fault。理解调用链即可，不要求手工实现完整编译器函数序言。

## Q38 MRS/MSR与MOV、LDR/STR有什么区别？

A 必学指令 | 一题一页，先理解再复述

**先读指南** 第3章p38~41、p47；第4章p56、p61。

**没懂再查** 第8章p133~134；官方02的核心寄存器访问接口。

**操作对象不同** MOV通常在通用寄存器与立即数之间传值；LDR/STR通过地址访问存储器或映射外设；MRS/MSR在通用寄存器和指定特殊寄存器之间传值。CONTROL、PSP等不按普通外设地址直接访问。

**两条指令记方向** MRS R0, PSP把PSP读到R0；MSR PSP, R0把R0写入PSP。不能只记名字相似，应从目标在左侧的实际操作数看方向。读取IPSR可帮助判断当前是否处在异常上下文。

**不是所有位都能随意写** 特殊寄存器受模式、权限和可写位限制。非特权代码不能通过随意写CONTROL把自己升级，写状态寄存器也不是把xPSR所有位都当普通RAM替换。

**在C中通常怎么用** 优先使用CMSIS提供的\_\_get\_PSP、\_\_get\_IPSR、\_\_get\_PRIMASK等接口。写SP或改变CONTROL应遵循端口/启动代码要求，必要时按规定配合屏障，不把示例直接塞进任意C函数。

**口述与自测** 问：LDR R0, =某寄存器名字一定能读取PSP吗？答：不能把符号加载伪指令当作特殊寄存器访问。用MRS或对应CMSIS读取，并确认当前上下文权限。

## Q39 从M3换到MSPM0，必须补哪些区别？

A 项目相关 | 一题一页，先理解再复述

**先读指南** 指南第3章p35~48、第7章p109~115：只用于复习M3对照基线。

**没懂再查** 本书不覆盖M0+。差异查官方01/02；MSPM0G3507查官方06产品页及其技术手册，不虚构本书对应页。

**共同点先复用** MSPM0G3507采用Cortex-M0+，仍可沿用MCU、寄存器、栈、向量和中断驱动的基础思路。换芯片不需要把GPIO、状态机、函数分层从零学一遍，但初始化库和外设路由必须重新核对。

**关键差异再补** M0+基于ARMv6-M，M3基于ARMv7-M；指令和异常功能不同。不能照搬BASEPRI、FAULTMASK、M3独占访问指令及完整Fault状态寄存器方案，经典M0/M0+也没有M3式优先级分组功能。

**工具与硬件各管一层** TI SysConfig负责生成配置，DriverLib提供外设操作接口；它们不是新CPU架构。实际定时器时钟、GPIO映射、中断号和引脚复用须看生成文件与器件手册，不从STM32的名字猜对应关系。

**端口为什么不能原样用** FreeRTOS端口包含指令、屏蔽和上下文相关实现，必须匹配内核与编译器。业务任务可能容易移植，底层端口和BSP不能只把工程CPU名称改一下就认为可靠。

**口述与自测** 问：M0+是不是把M3主频降低后的版本？答：不是，架构能力也不同。面试优先解释自己用到的定时器、五路GPIO和中断链，不夸大为所有TI外设均有深入经验。

## Q40 其他ARM学到哪？M4/M7与旧ARM怎样取舍？

B 对照；C 按岗深入 | 一题一页，先理解再复述

**先读指南** 指南第1章p15~19、第18章p247~252：架构背景与ARM7迁移对照。

**没懂再查** 本书不完整覆盖M4/M7。查官方07；A/R系列、TrustZone等仅在岗位要求时查对应内核文档。

**先守住主线** 你的准备方向是MCU/RTOS，M3应能说清启动、双栈、NVIC和上下文；M0+补项目相关差异。传统ARM的七模式、CPSR/SPSR和银行寄存器主要用于识别旧题适用范围，不作为M3标准答案。

**M4/M7需要认识什么** M4、M7基于ARMv7E-M，包含DSP扩展；浮点单元是否存在及精度能力要看具体内核配置和芯片。DSP中的SIMD是单指令处理多个数据元素，不是“一个周期执行很多任意指令”。

**M7新增什么风险** 带缓存的系统里，DMA与D-Cache一致性、缓存行对齐和存储区域配置会成为重要问题。不是简单认为主频高就可以忽略实时性；Cache命中、总线与外部存储等待还会影响时间行为。

**A/R怎么答够用** A通常面向复杂应用系统，R强调实时和可靠场景，M面向微控制器；这只是产品定位，不是A不能实时或只有R能做控制。暂不展开Linux、MMU页表和完整传统异常入口。

**口述与自测** 问：资料写ARM有37个寄存器，能直接回答STM32F103吗？答：不能，先确认处理器模型。准备深度跟岗位和本人项目走，不因题库出现旧概念就通读所有ARM架构。

## 阅读原文时，带着这份纠错清单

对应位置明确区分“华清原PDF”和“权威指南”

华清原PDF p1~2 ARM7/ARM9不能一概归为ARMv6。ARM7TDMI为ARMv4T；Cortex-M3为ARMv7-M，M4/M7为ARMv7E-M。

华清原PDF p3~5、p9 Thread不一定用PSP；PendSV是异常号14；MemManage=4、BusFault=5。复位先读MSP和复位向量，不是直接从0地址执行代码。

华清原PDF p6~7 堆增长方式不能一概而论；SIMD不等于单周期多指令；屏障不是简单强弱排序，参见Q30。

指南 p24、p48 PRIMASK不屏蔽NMI与HardFault；向量入口的Thumb最低位规则不适用于首项初始MSP，初始MSP反而应满足栈对齐。

指南 p52、p54 APSR饱和标志应为Q，不是S；p54的AND对应C按位与&，不是按位或|。看指令表时仍需核对具体指令。

指南 p72~73、p179~182 DMB不能当作全部访问完成屏障，ISB不是最强数据屏障；双栈/MPU示例不等于所有FreeRTOS任务都非特权运行。

指南 p206~207、p316 XN=1禁止取指、XN=0不因XN禁止取指，原文表格/解释有笔误；Fault示例不能占用RTOS已使用的PendSV，也不能盲目信任已损坏的栈。

这不是全书勘误 仅列本轮相关核查发现。旧工具链代码和寄存器配置不可无条件复制到当前工程。教材帮助理解，实际编码还要匹配内核、芯片和工具链。

# 官方查证入口

0编号对应每题的补充来源；标题可点击

中文主参考：Cortex-M3权威指南.pdf（Joseph Yiu著、宋岩译，322页中文版）。本版逐题标出的p均指该文件物理页。官方网页无固定页码，按所列函数或章节检索；版本不同请重新核对。

01 CMSIS: NVIC接口与不同内核的支持范围

02a CMSIS: CONTROL、MSP/PSP、PRIMASK/BASEPRI等

02b CMSIS: LDREX/STREX、DMB/DSB/ISB、WFI/WFE

03 ST PM0056: M3官方核对；XN见Rev7第116页

04 FreeRTOS GCC ARM\_CM3端口：函数名检索，匹配工程版本

05 FreeRTOS官方内核书：任务与中断等机制

06 TI MSPM0G3507: Datasheet与Technical documentation入口

07a ST PM0214: Cortex-M4编程模型与指令

07b ST AN4839: M7系列Cache与DMA，重点第3节

08 STM32F103C8资料入口：查RM0008的PWR/RCC/RTC章节

02包含02a/02b，07包含07a/07b。04在线main会更新，它只用于核查原理；工程事实以自己的实际端口版本为准。指南没有M0+/M4/M7直接讲解，相关指南页仅是对照基础。